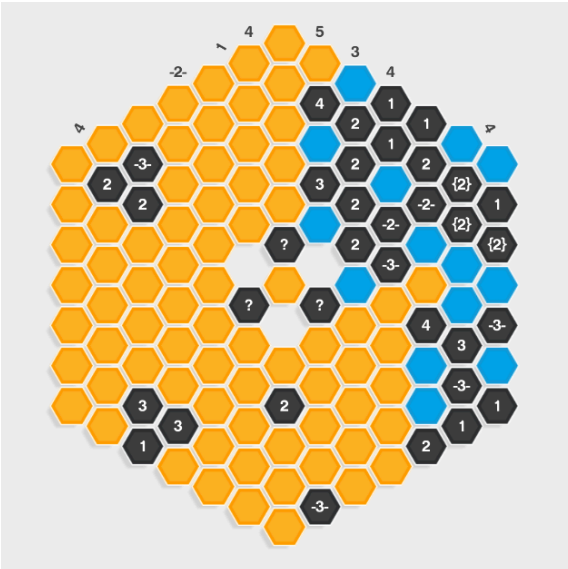
Problem



My project is focused on comparing automated solution techniques for the logic-puzzle game Hexcells. Hexcells combines the hidden mines aspect of Minesweeper with the grid-based constraints of Sudoku, all set on a hexagonal grid. Unlike Minesweeper, it is designed so that every move can be deduced through pure logic without ever needing to guess. Unmarked cells are yellow, marked mines are blue, and every other active cell is a black cell that gives you information about surrounding cells. For example, a black cell with a "2" in it tells you that 2 of the adjacent cells are mines. There are some additional rules; if a black cell has a "{2}", then 2 of the adjacent cells are mines and they are adjacent to each other. Conversely, if a black cell has a "–2-", then 2 of the adjacent cells are mine and they are not adjacent to each other. In addition, there are numbers along the edges of the puzzle that tell you how many mines are in that corresponding "column". There are additional rules, but I will refrain from explaining them all to keep this brief. Just know that there are many constraints to enforce that make solving this game more interesting than Minesweeper or Sudoku alone.

Approach, Baselines, & Data

I'm going with Track 3; I want to implement a few solution methods for this game. A few ideas I have so far include solving with an IP formulation in Gurobi, a SAT/SMT formulation in Z3, or seeing how far a heuristic-based approach can take me. I'm not sure which one of these I would use as the baseline, but I am aware of a solver on GitHub that I could compare my own methods to, even if they aren't as effective. For my data, Hexcells has its own seed-based level generator, so ideally I could create something that can process the GUI and convert the constraints into something that can be used with a solver or to inform a heuristic. Then I would need some way to control keyboard/mouse inputs to play the game and reveal more information. I could test various solving methods on a test set of seeds.

Timeline

Phase / Week	Deliverables & Tasks
Week 1 (Apr 6)	Project setup, data collection, and baseline environment configuration.
Week 2 (Apr 13)	Initial implementation of the core approach and preprocessing pipeline.
Week 3 (Apr 20)	Running baseline experiments and generating preliminary results.
Week 4 (Apr 27)	Iterative refinement of the model/approach based on initial metrics.
Week 5 (May 4)	Complete full experimental suite and begin data visualization.
Week 6 (May 11)	Final analysis, error checking, and polishing the documentation.
Final Due	**May 18: Project Submission**